

FyreWay VPN SDK: Production-Grade Infrastructure in Minutes



The complete architecture for device activation, server discovery,
and connection lifecycle management—without building the backend.

SDK v1.1.0 | Min Android: API 21 | Coroutines-First | Kotlin 1.9.21+

The Architecture of Abstraction

What We Handle (FyreWay SDK & Backend)

Device API (Tokens/Refresh)

Server API (Discovery/Caching)

Smart Connect Engine

Profile Provisioning

Analytics

What You Bring (Your App Layer)

Your UI (Compose/XML)

Your ViewModel

Your VPN Tunnel Implementation
(wireguard-android, ics-openvpn)

The SDK is not a tunnel. It manages the complex credentialing, selection, and reporting lifecycle so you can simply plug the output into your preferred VPN protocol library.

The Initialization Flow

1. Initialization

```
Application.onCreate()  
    ->  
    FyreWaySDK.initialize{}
```

(Singleton access,
requires baseUrl)

2. Device Activation

```
activateDevice("1.0.0")
```

Returns deviceId
& tokens

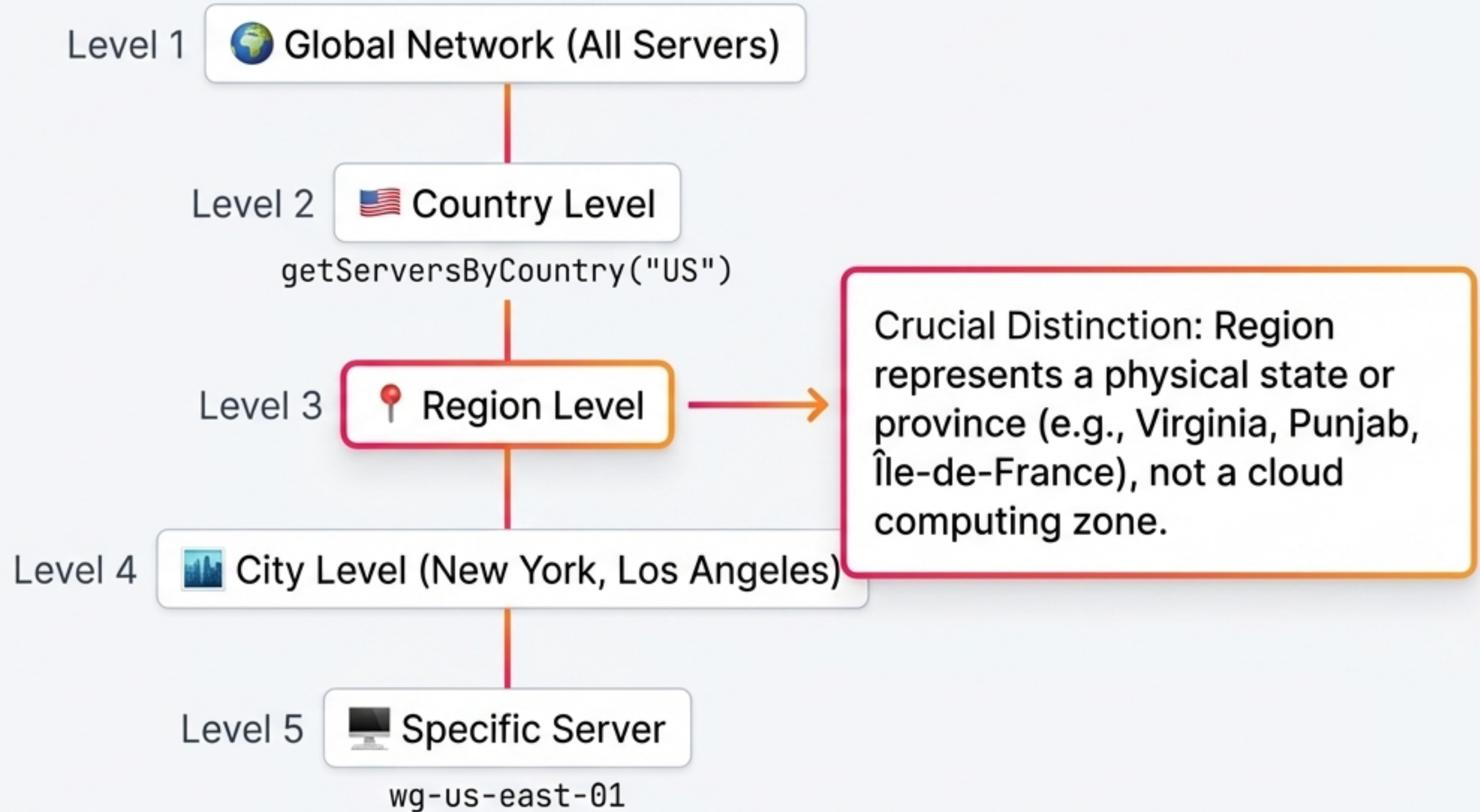
3. Proactive Resilience



The SDK securely stores credentials and runs a background job to automatically refresh tokens every 5 minutes before expiration.

Zero manual session management. Once activated, the device maintains its own authenticated state with the FyreWay backend indefinitely.

Global Location Hierarchy



Granular geographic routing allows developers to filter connection targets down to specific municipalities.

Server Selection: Manual vs. Smart

Option A - The Manual Path

< SELECT SERVER

Country: USA ▾

Region: Virginia ▾

City: Ashburn ▾

☀ us-east-01 (Loading...)

☀ us-east-02 (210ms)

☀ us-east-03 (180ms)

☀ us-east-02 (180ms)

CONNECT

- Fetch all servers
- Apply `ServerFilter.byCountry`
- Check `ServerCapacity`
- Compare `ServerPerformance` latency
- Present list to user
- User guesses best option

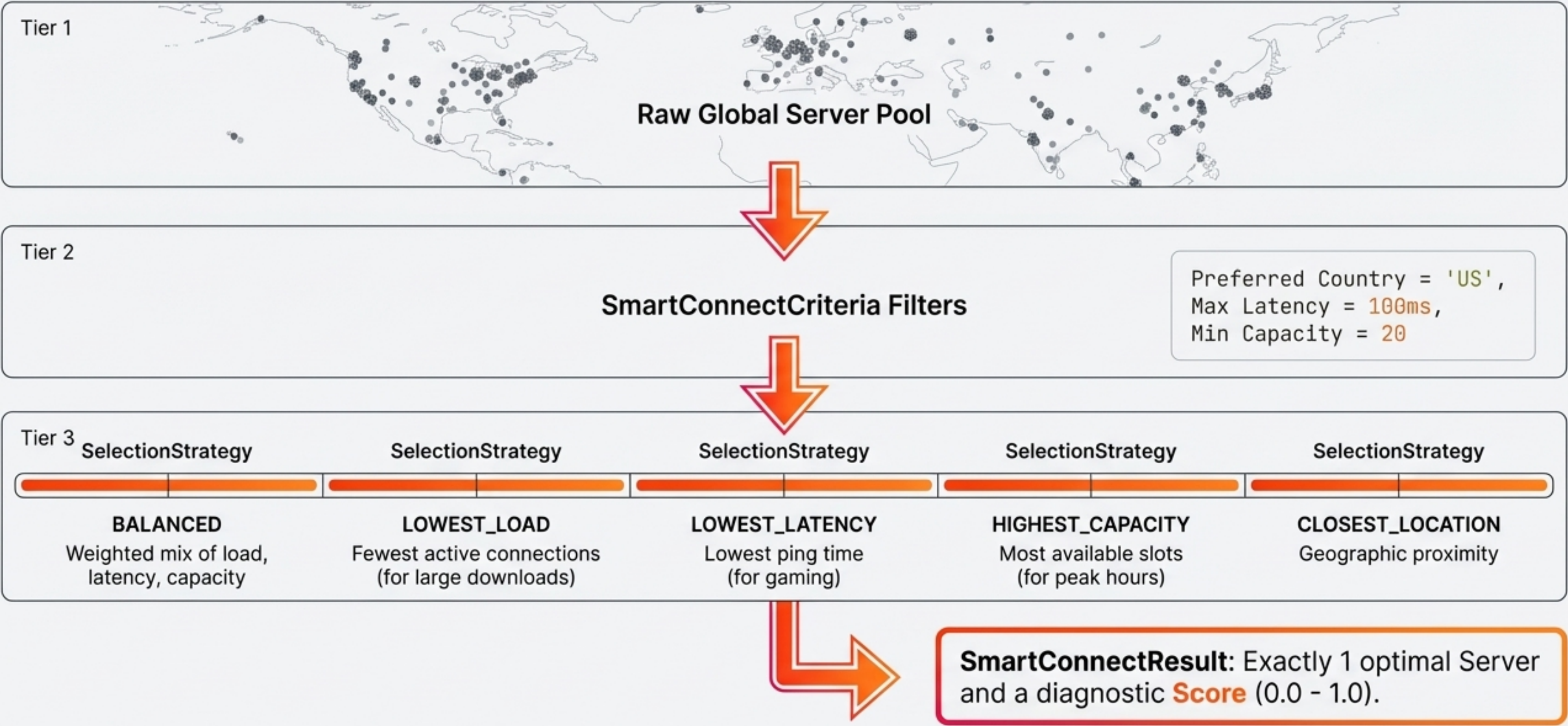
Option B - The Smart Abstraction

`smartConnect()`

Server
Profile

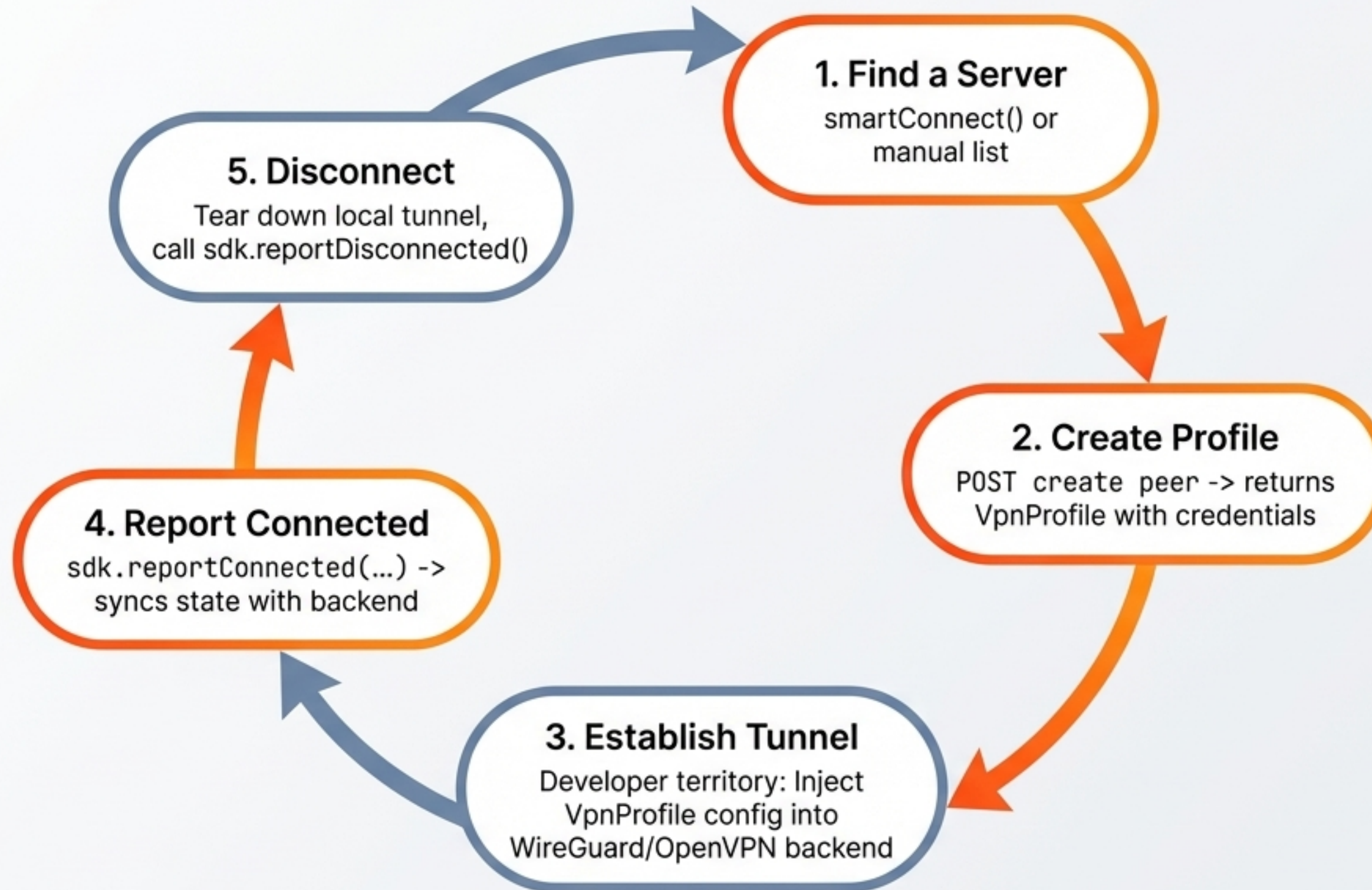
One-call optimal server selection. Backend load-balancing combined with edge-based scoring completely bypasses manual list parsing.

The Smart Connect Scoring Engine



The 5-Step Connection Loop

Orange = FyreWay Handled
Blue = Developer Handled

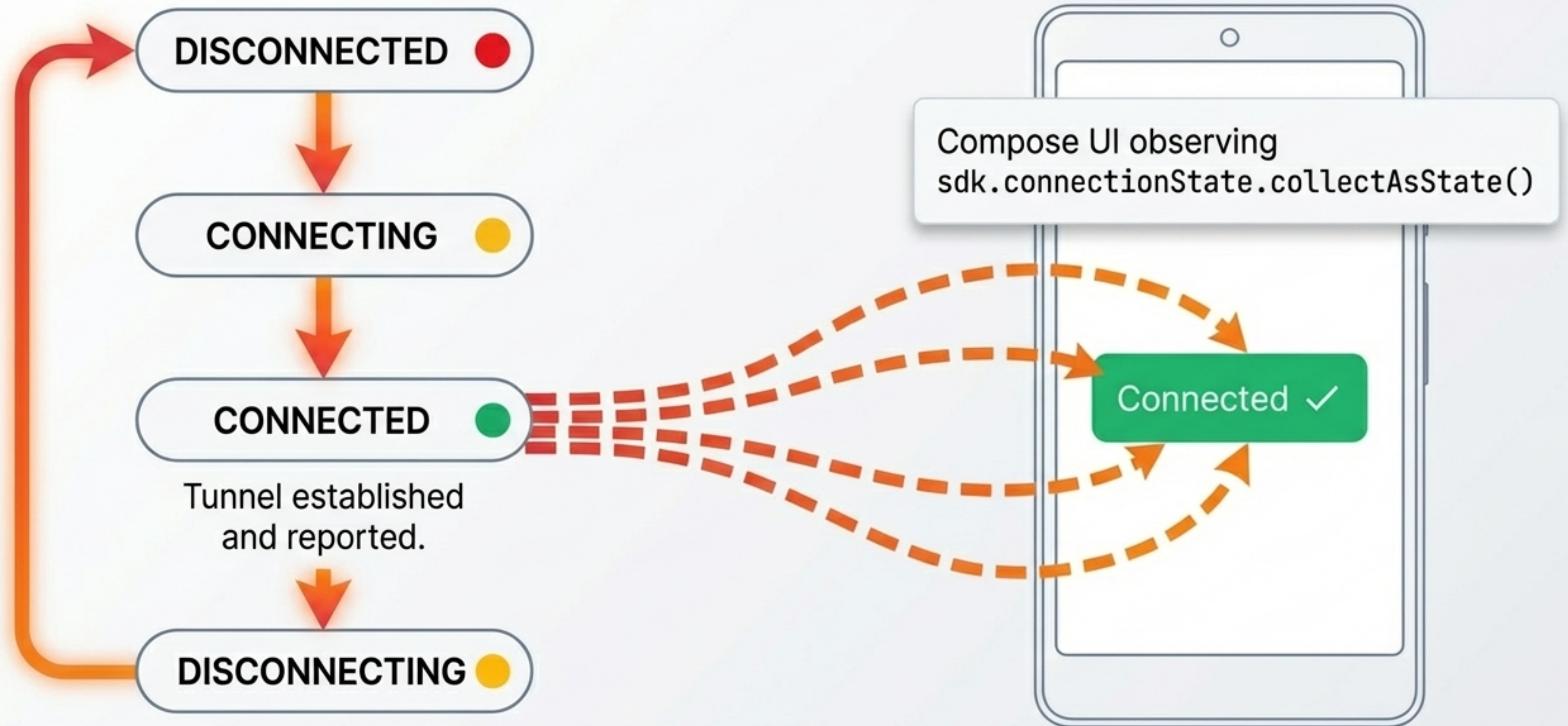


WireGuard vs. OpenVPN Protocol Integration

Both protocols are first-class citizens. `createProfile()` outputs the exact raw string required by your respective tunnel library.

Dimension	WireGuard	OpenVPN
Default Port	51820	1194
Config Extension	.conf	.ovpn
SDK Constant	VpnProtocol.WIREGUARD	VpnProtocol.OPENVPN
 The Peer Identifier origin	Uses the Client's own public key (profile.peerIdentifier). Must not be confused with serverPublicKey.	Uses the Client name / Certificate CN (profile.peerIdentifier).

Connection State Observability



UI Utilities & Data Formats

Country Flags

CountryFlags.getFlag('US')



Server Load Formatting



LOW
0-40%

MEDIUM
41-70%

HIGH
71-100%

Latency Levels

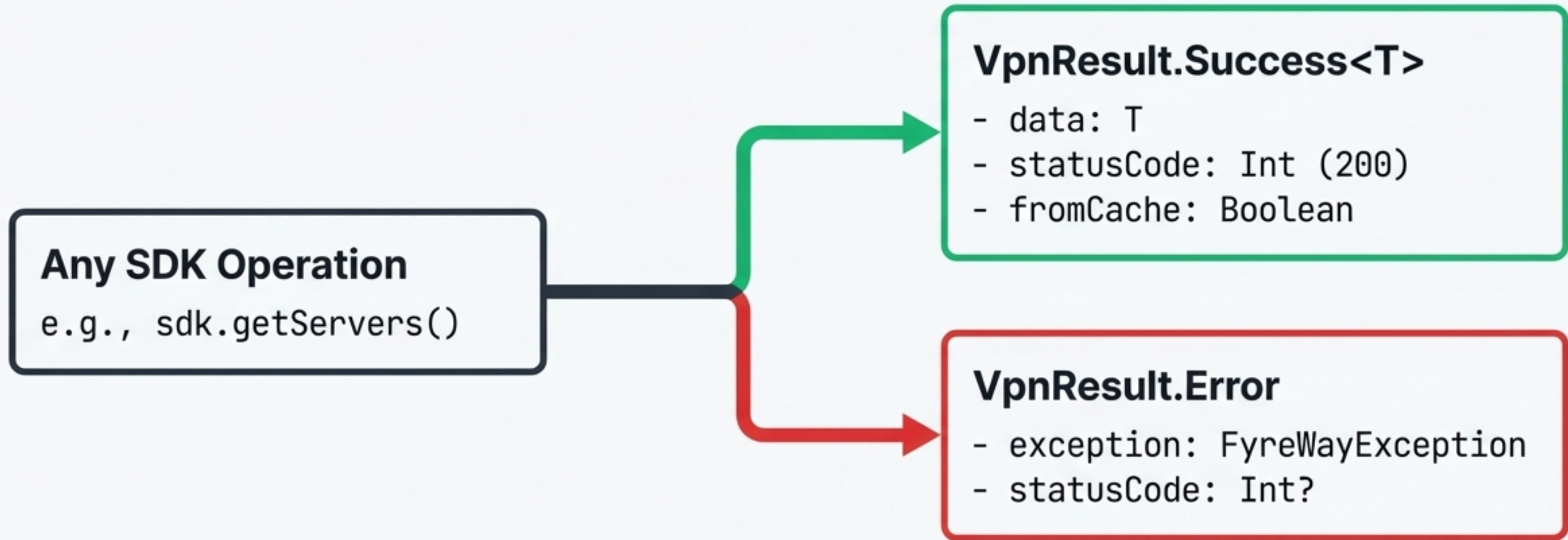
LatencyFormatter.format(25)



● Excellent	☰ Fair
📶 Good	● Poor

The **ServerListProvider** supplies complete **ServerDisplayItem** objects. Every field is ready to render directly to a Compose or XML view—zero manual formatting logic required.

The VpnResult Pattern



Predictable execution. There are zero unchecked exceptions in the FyreWay SDK. Every operation guarantees a type-safe Result, forcing exhaustive error handling at compile time and preventing silent UI failures.

Exception Handling Matrix

ErrorMessageMapper automatically converts these exceptions into user-friendly localized strings and titles.

Symptom	HTTP Code	Diagnosis	Exact Action
NetworkException	N/A	No internet/timeout	Show UI retry button.
AuthenticationException	401	Token expired/invalid	Re-call activateDevice(). (Do not assume tokens never expire).
RateLimitException	429	Too many requests	Wait retryAfterSeconds.
ProtocolNotSupportedException	N/A	Server rejects protocol	Filter for compatible servers.

Caching & Performance



Granular control over data freshness. The SDK automatically caches server lists and location hierarchies, but developers dictate the retrieval policy per call.

Synthesis: The Production Checklist

- ☐ **Security:** `enableLogging = false` in release builds.
- ☐ **Infrastructure:** `baseUrl` securely loaded from `BuildConfig` (Never hardcoded).
- ☐ **State Sync:** `reportConnected` and `reportDisconnected` strictly implemented to prevent ghost sessions on the backend.
- ☐ **UX Resilience:** All `VpnResult.Error` cases handled with user-friendly mapped messages.
- ☐ **Obfuscation:** Consumer ProGuard rules automatically applied (`com.fyreway.vpn.sdk.**`).

From zero to VPN. FyreWay manages the infrastructure complexity so you can focus on building the ultimate client experience.